

Claude Project Setup Prompt

Date: 2026-07-03

Status: copy-ready prompt for onboarding Claude Code to the project, skills, MCPs, and local checks.

Copy This Prompt Into Claude Code For Setup

You are Claude Code being onboarded into the ArchFlow public-safe project. Your first task is setup and readiness, not feature implementation.

Do not edit feature files yet. Do not run provider calls. Do not write external systems. Do not send Telegram. Do not push. First, prove you understand the project, available skills, MCP/connectors, safety boundary, validation commands, and next execution sequence.

Setup Goal

Create a reliable local cowork setup so you can later execute ArchFlow tasks without breaking the knowledge system, dashboard, public-safety boundary, or live agent coordination.

Your setup output must be a concise readiness report and, only if useful, small project-safe setup notes. The setup must answer:

- Which project files define the operating contract?
- Which project-local skills must be read before work?
- Which MCP/connectors are available in this Claude environment?
- Which connectors are missing or gated?
- Which validation commands are available?
- Which runtime claims are proven locally and which are only planned?
- What is the correct next execution task?

Mandatory Safety Boundary

Public tracked files must not contain:

- API keys, tokens, cookies, passwords, or credential values;
- private Notion page URLs;
- account, workspace, project, deployment, chat, or user IDs;
- local absolute paths;
- raw transcripts, raw private documents, screenshots, browser logs, raw PDFs, or raw voice/audio;
- private source text;
- private profile data;
- unsupported customer, revenue, ROI, or production claims.

Use repo-relative paths in notes and commits.

Step 1 - Identify Repo State

Run:

```
git status --short --branch
```

Then inspect the current branch and whether the worktree is dirty. If there are existing changes, do not overwrite them. Treat them as user or prior-agent work unless clearly generated by your current run.

Step 2 - Read The Operating Contract

Read these files in this order:

```
project/operating-rules.md
project/live/communication/README.md
project/live/communication/current-agent-notice.md
project/live/communication/agent-communication-log.md
project/README.md
project/project-plan.md
project/autentic-stack.md
project/config/model-routing.yaml
project/agents/agent-roster.yaml
wiki/index.md
wiki/memory.md
wiki/insights.md
```

Read only the latest relevant sections of long logs first. Search if needed; do not dump long files into chat.

Step 3 - Register The Live Coordination Claim

Before any edit, append a starting entry to:

```
project/live/communication/agent-communication-log.md
```

Include:

- task;
- files likely to change;
- files claimed;
- expected output;
- blockers;
- next step.

If this is only a read-only setup audit, still append a start entry only if you will write a setup report. If no files will be edited, report read-only status in chat and do not claim broad folders.

Step 4 - Read Project-Local Skills

Read these skills as project operating contracts:

```
skills/arcagcom/SKILL.md
skills/task-handout/SKILL.md
skills/archflow-e1-runtime-guard/SKILL.md
skills/archflow1/SKILL.md
skills/priority-task-operator/SKILL.md
skills/archflow-task-breakdown/SKILL.md
skills/outquestions/SKILL.md
skills/daily-public-project-review/SKILL.md
skills/evening-skill-registry-update/SKILL.md
```

For each skill, record:

- when to use it;
- required files;
- required checks;
- stop rules;
- whether it applies to the current task.

If Claude Code supports a local project skill system, adapt these skills by registering or mirroring them only into Claude's project-local skill mechanism. Do not install external skills from the network. Do not copy secrets. If Claude Code has no skill installation mechanism, keep them as required project docs and cite them in your readiness report.

Step 5 - Discover MCPs And Connectors

Use Claude Code's normal MCP discovery commands or UI. Do not guess tool names.

For every available MCP or connector, record:

- server/connector name;
- tool list source;
- whether schemas were inspected;
- what it can safely do in this project;
- whether it requires approval before writes;
- whether it is currently usable or missing.

Expected connector categories to check:

- GitHub;
- Notion;
- Obsidian/Nexus;
- Figma;
- Vercel;
- Railway;
- Telegram or sender scripts;
- OpenRouter or provider bridge;
- local filesystem/search;
- browser or dashboard testing tools.

Important connector rules:

- GitHub: no force push, no history rewrite, no push unless approved.
- Notion: append evidence or propose status changes; do not mark Done without proof and approval.
- Obsidian/Nexus: use only after tool/schema discovery and correct vault/runtime confirmation.
- Figma: only for explicit design/deploy-sync tasks.
- Vercel: static/hosted proof only; no production promotion unless approved.
- Railway: gated always-on backend lane; not required for static dashboard proof.
- Telegram: use approved sender scripts and ignored env only; store sanitized delivery status only.
- OpenRouter: disabled until explicit provider approval, fresh model list, sanitized packet, budget cap, ledger, and reviewer.

If a connector is unavailable, write:

GAP: <connector> unavailable or not configured; fallback is <filesystem/local script/manual review>.

Step 6 - Verify Local Validation Commands

Run the lightweight setup checks:

```
python3 scripts/public_safety_scan.py
```

```
project/local/venv/bin/python project/scripts/validate-workflows.py
project/local/venv/bin/python project/scripts/pre-push-runtime-guard.py
python3 project/scripts/generate-dashboard-data.py
python3 -c 'import json; json.load(open("project/dashboard/data.json")); print("dashboard_json_ok")'
git diff --check
```

If the task will later touch dashboard UI, also check whether this command is available:

```
project/local/venv/bin/python project/scripts/dashboard-static-smoke.py
```

If it fails because local loopback binding is blocked, record that as an environment permission issue, not a dashboard defect.

If the task will later touch RAG, run:

```
project/local/venv/bin/python project/scripts/llamaindex-approved-corpus.py --mode smoke
project/local/venv/bin/python project/scripts/llamaindex-rag-benchmark.py
```

Do not install packages during setup unless explicitly approved.

Step 7 - Confirm Runtime Claims

Use this truth table:

Layer	Current working assumption
Codex/local operator	Primary editor/reviewer boundary.
WikiLLM	Durable public-safe memory in wiki/.
Live communication	Required coordination channel in project/live/communication/.
LlamaIndex	Bounded hybrid retrieval with lexical fallback over approved public corpus.
LangGraph	Workflow controller contract and smoke path.
CrewAI	Role/task contract and gated runtime proof, not default autonomous runtime.
Dashboard	Static/browser-local command surface plus hosted provider-disabled review packets.
Jarvis	Review-packet operation only unless backend/provider proof is separately verified.
Website	Static public positioning surface for PRD/ICP product-team direction.
Notion	Task/reporting surface, not public source of truth in repo artifacts.
Telegram	External send path only through approved sender and sanitized status.
OpenRouter	Disabled until explicit approved provider lane.
Railway	Future always-on backend lane, gated.
Nexus	Future/live Obsidian bridge only after schema discovery and verified tool calls.
Vector/turbovec	Deferred until embedder/vector readiness, stable IDs, metadata, source filters, and benchmark proof.

If you find evidence that changes any row, cite the file or command proof and update the readiness report. Do not promote a planned layer to implemented state without current proof.

Step 8 - Produce Setup Readiness Report

Create or update a setup report only if the user asked for a durable artifact. Suggested path:

```
project/runs/<date>-claude-setup-readiness/agent-handout.md
```

For this current continuation lane, if only updating the existing handoff packet, write under:

```
project/runs/2026-07-03-claude-continuation-stabilization/
```

The report must include:

- files read;
- skills read and applicability;
- MCP/connectors discovered;
- connectors missing or gated;
- validation commands run and results;
- current runtime truth table;
- next recommended task;
- stop conditions;
- exact next safe action.

Step 9 - Setup Done Criteria

Setup is complete only when:

- you can explain the project structure without chat history;
- you know the safety boundary;
- you know which skills to read for each work type;
- you know which MCPs are available and which are gated;
- you know how to validate changes;
- you know that the next execution sequence is E1.4, E2.0A, E3.1, E4.1, then TG cleanup;
- you have not installed external packages, called providers, sent Telegram, changed Notion, deployed, or pushed without approval.

Setup Output Format

Return:

```
FACT:
- ...

INTERPRETATION:
- ...

GAP:
- ...

READY / NOT READY:
- ...

NEXT SAFE ACTION:
- ...
```

Keep the chat response concise. Put detailed durable output into files.